The Journal of Open Source Software

GHOST: A C++ framework to model and solve combinatorial optimization problems

Florian Richoux [1¶]

1 Artificial Intelligence Research Center, National Institute of Advanced Industrial Science and Technology (AIST), Japan ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Fangzhou Xie

Reviewers:

- @alexzerg
- @cprudhom

Submitted: 14 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

GHOST (General meta-Heuristic Optimization Solving Toolkit) is an open-source C++20 framework for modeling and solving discrete combinatorial problems (Richoux, 2014-2026). It supports Constraint Satisfaction Problems (CSPs), Constrained Optimization Problems (COPs), Error Function-based Constraint Satisfaction Problems (EF-CSPs) and Error Function-based Constrained Optimization Problems (EF-COPs) (Richoux et al., 2016; Richoux & Baffier, 2023). Users declare variables, constraints, and, optionally, an objective function, while GHOST supplies a constraint-based local search solver.

GHOST was initiated in 2014 for a *StarCraft: Brood War* bot, where all decisions for a game frame had to be computed in less than 100 ms. It has since supported research on decision making with short time budgets, terrain analysis, automatic error-function learning, Quadratic Unconstrained Binary Optimization (QUBO) reformulation, and combinations of combinatorial optimization with Monte Carlo Tree Search (Antuori & Richoux, 2019; Richoux, 2022, 2025; Richoux et al., 2023, 2016; Richoux & Baffier, 2023). Its benchmark repository includes classic combinatorial (optimization) problems, such as Sudoku, N-Queens, the knapsack problem, magic squares, the quadratic assignment problem, the traveling salesman problem, the traveling tournament problem, and vertex cover (Richoux, 2026).

Statement of need

Combinatorial optimization software involves trade-offs between modeling convenience, control over search, solution quality, and response time. Complete solvers, i.e., solvers exploring the whole search space, are valuable when optimality or infeasibility proofs are required, but such guarantees may be incompatible with interactive applications. Conversely, implementing a specialized metaheuristic provides control but requires substantial effort and often entangles the problem model with the search procedure.

GHOST addresses the intermediate need for a reusable native C++ framework that accepts constraint-based models and provides a ready-to-use metaheuristic solver. Its intended users are researchers and C++ developers who need feasible and improving solutions under limited computation budgets, wish to test alternative models without repeatedly implementing a solver, or need to embed combinatorial decision making in a C++ system. Relevant domains include decision-making, games, scheduling, logistics, routing, assignment, and resource allocation.

Constraints in GHOST have non-negative error functions, for which zero denotes satisfaction. An error function is very similar to a penalty function, with the difference that error functions express hard constraints, whereas penalty functions express soft constraints. The solver therefore obtains information about how far a candidate is from feasibility rather than only a Boolean signal. Because Adaptive Search is a metaheuristic, GHOST is an anytime solver:

DRAFT

it returns its best solution when the allocated time expires, but cannot prove optimality or infeasibility.

State of the field

Constraint programming systems such as Choco (Prud'homme & Fages, 2022) and OR-Tools CP-SAT (Google, 2010-2026) provide expressive models and systematic solving techniques. They are natural choices when strong propagation, bounds, or proof-oriented solving are principal requirements. Comet introduced language-level support for constraint-based local search (Van Hentenryck & Michel, 2005), while OscaR.cbls provides a Scala framework for composing models, invariants, neighborhoods, and search procedures (CETIC, 2018-2026). Among other framworks and solvers written in C++, we can find Gecode (Schulte & Stuckey, 2008), EasyLocal++ (Ceschia et al., 2024) and ParadisEO (Dreo et al., 2021), the latters being also based on metaheuristics. The main difference between GHOST and EasyLocal++ or ParadisEO is that GHOST is a framework only asking users to model their problem, with a complete automation of the solving part, when both EasyLocal++ and ParadisEO requires users to explicitely code their solvers. GHOST's solver has been designed in a modular way to allow deep modifications of its algorithm, but this was developed for research purposes and is not intended for typical users. Another important difference between GHOST and other constraint-based frameworks and solvers is that GHOST is the only one supporting (hard) constraints expressed as error functions, to the best of our knowledge. Finally, GHOST includes a constraint-based local search solver, though it employs an approach that differs from that of Comet, making the definition of problem models in GHOST more intuitive for developers: the only mechanism users need to provide are error functions, that is, functions that calculate the extent to which a given assignment violates each constraint. Providing delta error functions - which compute the error difference between two assignments - are encouraged to get better permormances, but are not mandatory, unlike in Comet.

The Constraint programming community mainly proposes different languages to declare problem models, such as XCSP3 and MiniZinc, and compiled a large catalogue of more than 400 global constraints that users are encouraged to use in order to achieve the best solver performance. Originally, these global constraints aimed to isolate the most frequently used constraints and optimize their internal mechanisms, so that users would not have to reinvent the wheel and would have a comprehensive set of various constraints at their disposal to express their problems. However, global constraints are mainly useful for complete solvers since they implement complete solver-specific mechanisms such as propagation, which are useless for meta-heuristics. Moreover, searching for the right constraints in a +400 global constraints catalog can be a daunting task for users/developpers, in particular when they know what combination of values are possible or not in their problem, and can express that easily with code.

This is why `GHOST` occupies a different design point from other tools in the field. It is a lightweight C++ framework intended for direct integration into native applications and for problems with short, explicit time budgets. Users provide a constraint-level model while the framework supplies generic search control. This reduces the need to implement neighborhood exploration and metaheuristic logic for every application, while retaining extensibility through custom constraints and objective functions. Its priorities are ease of modeling and use, then execution speed. Users can define their own, custom constraints, but also use predefined constraints among a very restricted set of the most common ones. For the moment, only AllDifferent, Linear Equation, and their variants, are implemented. `GHOST` therefore complements rather than replaces exact solvers and more configurable local-search toolkits.

DRAFT

The Journal of Open Source Software

Software design

The main GHOST solver implements Adaptive Search, a constraint-based local search algorithm (Codognet & Diaz, 2001). From a current assignment, its neighborhood contains every candidate reachable by changing one variable's value. It evaluates the constraint error functions and projects onto each variable the errors of the constraints it is involved in. The search for solutions is guided by changing the assignment of variables with the largest error. After finding a solution, that is, an assignment satifying all constraints of the problem, it seeks better solutions according to the objective. When search appears trapped in a local optimum, it performs either a reset, randomly reassigning some variables, or a restart, randomly reassigning all variables.

The architecture separates problem models from search, rooted in the philosophy of the "holy grail" of Constraint Programming: "Users declare the problem, the computer solves it." A model defines variables and domains, constraints, and an optional objective. Satisfaction and optimization share these abstractions, and permutation and non-permutation representations are supported.

In addition to the constraint-based local search solver, a simple, complete solver based on depth-first search with AC3 filtering, was later added to enumerate all solutions of an instance. This was motivated by the AI agent playing the board game *boop.* on an Android app, where enumeration was needed within Monte Carlo Tree Search (Richoux, 2025). This simple complete solver can avoid complex backtrackings since it aims for finding all solutions, so if a computation leads to a dead-end, no real backtrack management is necessary: The solver simply continues its depth-first search via recursive calls. The framework's evolution is documented in its change log.

GHOST is built with CMake and distributed under GNU GPL v3. The repository includes installation instructions for GNU/Linux, macOS, and Windows, tutorials, an introduction to constraint programming, Doxygen API documentation, automated tests, continuous integration, releases, a PDF cheat sheet, and a 30-minute introductory video (Richoux, 2014-2026, 2021). A separate repository provides executable benchmarks (Richoux, 2026).

Research impact statement

GHOST has been developed openly since 2014. The original work modeled target selection, wall-in construction, build-order planning, and resource allocation for real-time strategy games (Richoux et al., 2016). Further work studied robustness and flexibility on pathfinding and alternative resource-allocation models (Fradin & Richoux, 2015). For the time being, GHOST is not yet widely distributed and remains used primarily by its creator for research purposes, in different contexts.

The software has subsequently enabled research on decision making under uncertainty and the `microPhantom` bot for *microRTS* (Antuori & Richoux, 2019; Richoux, 2020), constraint-based terrain analysis in *StarCraft* 1 and 2 (Richoux, 2022), learning interpretable error functions (Richoux & Baffier, 2021, 2023), learning QUBO constraint representations (Richoux et al., 2023), and injecting combinatorial optimization into Monte Carlo Tree Search for *boop.* (Richoux, 2025).

GHOST is integrated into `pobo`, an Android implementation of *boop.*; `Taunt`, a C++ terrain-analysis library for *StarCraft* 1 and 2; and `microPhantom`, a bot for *microRTS*. It has also supported research systems for learning constraint error functions and QUBO reformulations. These applications influenced its evolution, including complete solution enumeration.

DRAFT

AI usage disclosure

Microsoft 365 Copilot was used to prepare and revise this manuscript from the repositories, JOSS guidance, bibliographic sources, and information supplied by the author. The author reviewed the text and remains responsible for it. No generative AI was used to produce the GHOST source code or documentation.

Acknowledgements

GHOST was developed while the author held positions at the French National Centre for Scientific Research (CNRS), Nantes University, and the National Institute of Advanced Industrial Science and Technology (AIST). These organizations did not provide dedicated funding specifically for GHOST.

References

Antuori, V., & Richoux, F. (2019). Constrained optimization under uncertainty for decision-making problems: Application to real-time strategy games. *IEEE Congress on Evolutionary Computation*. https://doi.org/10.1109/CEC.2019.8789922

Ceschia, S., Da Ros, F., Di Gaspero, L., & Schaerf, A. (2024). EasyLocal++ a 25-year perspective on local search frameworks: The evolution of a tool for the design of local search algorithm. *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 1658–1667. https://doi.org/10.1145/3638530.3664140

CETIC. (2018-2026). *OscaR.cbls*. https://github.com/cetic/oscar-cbls

Codognet, P., & Diaz, D. (2001). Yet another local search method for constraint solving. *Stochastic Algorithms: Foundations and Applications*, 73–90. https://doi.org/10.1007/3-540-45322-9_5

Dreo, J., Liefooghe, A., Verel, S., Schoenauer, M., Merelo, J. J., Quemy, A., Bouvier, B., & Gmys, J. (2021). ParadisEO: From a modular framework for evolutionary computation to the automated design of metaheuristics: 22 years of ParadisEO. *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 1522–1530. https://doi.org/10.1145/3449726.3463276

Fradin, J., & Richoux, F. (2015). Robustness and flexibility of GHOST. *AIIDE Workshop on Artificial Intelligence for Strategy Games*. https://doi.org/10.1609/aiide.v11i2.12809

Google. (2010-2026). *OR-Tools*. https://developers.google.com/optimization/

Prud'homme, C., & Fages, J.-G. (2022). Choco-solver: A Java library for constraint programming. *Journal of Open Source Software*, *7*(78), 4708. https://doi.org/10.21105/joss.04708

Richoux, F. (2025). Injecting combinatorial optimization into MCTS: Application to the board game boop. *Applications of Evolutionary Computation*, *15613*, 525–540. https://doi.org/10.1007/978-3-031-90065-5_32

Richoux, F. (2022). Terrain analysis in StarCraft 1 and 2 as combinatorial optimization. *IEEE Congress on Evolutionary Computation*, 1–8. https://doi.org/10.1109/CEC55065.2022.9870230

Richoux, F. (2014-2026). *GHOST: General meta-heuristic optimization solving toolkit*. https://github.com/richoux/GHOST

Richoux, F. (2020). microPhantom: Playing microRTS under uncertainty and chaos. *IEEE Conference on Games*. https://doi.org/10.1109/CoG47356.2020.9231653

DRAFT

Richoux, F. (2021). *Introduction to GHOST*. YouTube video. https://youtu.be/GyIF0125KZQ

Richoux, F. (2026). *GHOST Benchmarks*. https://github.com/richoux/GHOST-benchmarks

Richoux, F., & Baffier, J.-F. (2021). Error function learning with interpretable compositional networks for constraint-based local search. *Genetic and Evolutionary Computation Conference Companion*, 137–138. https://doi.org/10.1145/3449726.3459464

Richoux, F., & Baffier, J.-F. (2023). Automatic error function learning with interpretable compositional networks. *Annals of Mathematics and Artificial Intelligence*, *93*, 441–469. https://doi.org/10.1007/s10472-022-09829-8

Richoux, F., Baffier, J.-F., & Codognet, P. (2023). Learning QUBO models for quantum annealing: A constraint-based approach. *Computational Science – ICCS 2023*, *14077*, 153–167. https://doi.org/10.1007/978-3-031-36030-5_12

Richoux, F., Uriarte, A., & Baffier, J.-F. (2016). GHOST: A combinatorial optimization framework for real-time problems. *IEEE Transactions on Computational Intelligence and AI in Games*, *8*(4), 377–388. https://doi.org/10.1109/TCIAIG.2016.2573199

Schulte, C., & Stuckey, P. J. (2008). Efficient constraint propagation engines. *Transactions on Programming Languages and Systems*, *31*(1), 2:1–2:43. https://doi.org/10.1145/1452044.1452046

Van Hentenryck, P., & Michel, L. (2005). *Constraint-based local search*. MIT Press. https://doi.org/10.1007/978-3-032-00385-0_7

DRAFT